

Q.1) Write a C program to find whether a given files passed through command line arguments are present in current directory or not.

```
#include <stdio.h>
```

```
#include <sys/stat.h>
```

```
int main(int argc, char *argv[]) {
```

```
    if (argc < 2) {
```

```
        printf("Usage: %s <file1> <file2> ...\n", argv[0]);
```

```
        return 1;
```

```
    }
```

```
    struct stat fileStat;
```

```
    for (int i = 1; i < argc; i++) {
```

```
        if (stat(argv[i], &fileStat) == 0) {
```

```
            printf("File '%s' exists in current directory.\n", argv[i]);
```

```
        } else {
```

```
            printf("File '%s' does NOT exist in current directory.\n", argv[i]);
```

```
        }
```

```
    }
```

```
    return 0;
```

```
}
```

Q.2) Write a C program which creates a child process and child process catches a signal SIGHUP, SIGINT and SIGQUIT. The Parent process send a SIGHUP or SIGINT signal after every 3 seconds, at the end of 15 second parent send SIGQUIT signal to child and child terminates by displaying message "My Papa has Killed me!!!".

```
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <time.h>

void handle_signal(int sig) {
    switch(sig) {
        case SIGHUP:
            printf("Child received SIGHUP\n");
            break;
        case SIGINT:
            printf("Child received SIGINT\n");
            break;
        case SIGQUIT:
            printf("My Papa has Killed me!!!\n");
            exit(0);
    }
}

int main() {
    pid_t pid = fork();

    if (pid < 0) {
        perror("fork failed");
        exit(1);
    }

    if (pid == 0) {
        // -----
```

```

// Child process: set up signal handlers
// -----
signal(SIGHUP, handle_signal);
signal(SIGINT, handle_signal);
signal(SIGQUIT, handle_signal);

printf("Child process started with PID = %d\n", getpid());

while(1) {
    pause(); // Wait for signals
}
} else {
    // -----
    // Parent process: send signals
    // -----
    int signals[5] = {SIGHUP, SIGINT, SIGHUP, SIGINT, SIGQUIT};

    for (int i = 0; i < 5; i++) {
        sleep(3);
        kill(pid, signals[i]);
        printf("Parent sent signal %d to child\n", signals[i]);
    }

    printf("Parent exiting.\n");
}

return 0;
}

```